

BIGNUMBERS.PY DOCUMENTATION

Merlijn de Smit

Requirements

Dr. Frankenstein at the Astronomy Department gathers data on the carbon content of distant stars. He has become preoccupied with the notion that these data, expressed in absolute terms, represent prime numbers more often than one would expect. No one knows what this would mean, or why it would be interesting or relevant, but Dr. Frankenstein has increasingly become obsessed with the idea. He has now requested the Computer Department to provide him with a script that can quickly determine the lowest prime factor of a given large number.

The script should therefore:

- Be able to take a list (of arbitrary length) of numbers (of arbitrary size) as input, and output a list of their lowest prime factors (which can be equal to the number itself, in which case it is prime). The output should be saved as a file.
- Be able to elegantly handle problems of user input with error messages, instead of the script itself aborting, Dr. Frankenstein does not like coding himself – he wants errors to be reported in human-readable language as much as possible.

Design

The script uses the following modules/libraries:

- numpy, in order to use the array type
- csv, in order to write to a .csv file
- pyplot from matplotlib, in order to plot a bar diagram
- sys, in order to quit the script upon encountering a specific error

The script should be structured as follows:

- A **function** that determines the lowest prime factor of a given number by calculating the modulo of the number and a variable *i* that increments from 2 to the size of the number itself (1 is excluded as any number is divisible by one). As soon a modulo of zero is found, the associated value of *i* is the lowest prime factor of the number in question.
- **Data input** by the user either manually entering a number or providing the name of a file containing numbers (which should be present in the same directory as the script):
 - The possibility to **manually enter a number** is mainly there for testing purposes.
 - When a **file name** is given, the script should read the file and perform an elementary check on the presence of problematic (non-numeric) characters and quit if such characters are found.
 - If the file does not contain problematic characters, the script will read the numbers as an array and apply the function mentioned above to each element in it. The results are appended to an empty list, and eventually the two lists (the numbers and the results/factors) are "zipped" into a Python dictionary. This dictionary is **written into an external .csv file** using the csv module.
 - Optionally, a **bar diagram** can be produced depicting the frequencies of various factors (e.g. "prime", "3", "2", etc.). This makes sense only for bigger files which show variation in terms of their lowest factors, and is therefore an optional (not a standard) output.
 - This is achieved by creating a list containing only the unique values of the original results list (i.e. by turning the list into a set and then back into a list), and then by

looping each of the unique values through the original results list in order to determine how often each unique value occurs there. Results (unique values and frequencies) are written into a Python dictionary. The keys and values of this dictionary are then plotted into a bar diagram using the pyplot function of the matplotlib library.

Problems: the only problem encountered in implementing the script above relates to the diagram function. Initially, both the x-axis and the y-axis were unordered. The x-axis was ordered by sorting the list of unique values (and temporarily changing "Prime" to "0" in order to enable sorting). Solving the issue for the y-axis took some time until it was determined that the y-axis values were read as strings rather than integers. Once this was solved, the bar diagram was outputted correctly.

Verification

The script was tested with the following files as input:

- largenumbers.txt, a list of 90 random large numbers (generated by ChatGPT). This was used to test whether results were outputted correctly as a .csv file and a bar diagram (not necessarily whether the results themselves were correct).
- smallnumbers.txt, a small list of small numbers with immediately obvious lowest prime factors. This was used to test whether the script correctly determined lowest prime factors.
- allprimenumbers.txt, a list of 20 large prime numbers (generated by ChatGPT). This was used to additionally test whether lowest prime factors were determined correctly by the script.
- largenumbers_error.txt, a list of 90 random large numbers with a few errors/forbidden characters. This was used to test the testing for forbidden characters in the script.

Main problems encountered during testing related to the bar diagram (see above). In its current implementation, the bar diagram and all other outputs are handled correctly by the script.

At various points in the script, print()-calls for outputting intermediate results to the screen are commented out, but can be de-commented if desired for testing or debugging purposes.

Remaining problems and questions

The program does not currently handle situations where the data file cannot be found (except by violently quitting). An error message for this situation would be nice to have in the future.

The script is currently fastidious about the data it accepts: only files containing numbers, whitespaces and line breaks will be accepted. Adding more scripting for data cleaning (currently, only line breaks are cleaned) would perhaps allow the script to accept a wider variety of data.

The outputted .csv file contains the data, but no headers. These would have to be added manually.

The outputted .csv file will get overwritten any time the script is run (which can be avoided by moving or renaming the .csv file). In future development, adding a timestamp or perhaps a random number to the file name could avoid this.

AI disclaimer

ChatGPT was used in obtaining number lists for testing, but not in developing the code itself. This was done entirely manually (and by googling e.g. StackOverflow for troubleshooting).